

Nombres Completos:

Josseline Jhomayra Granda Chacha

Materia:

Herramientas Informáticas para el Despliegue de Diagramas

Ciclo/Carrera:

3ro Ciclo de Desarrollo de Software “A”

Tema:

Base de datos con arrays

Fecha de entrega:

10/10/2026

Docente:

ING. Jhostin Vacacela

Ciudad:

Cuenca

1. Objetivo

El objetivo fue crear y revisar una API en NestJS que utiliza un array para guardar usuarios temporalmente. Se corrigieron errores de TypeScript, se revisaron el servicio y los DTO, y se prepararon pruebas en Postman para conocer las operaciones CRUD y otros métodos HTTP.

2. Registro de las consultas realizadas

Las consultas se presentan en el orden de la conversación, con su objetivo, la información obtenida y su relación con el siguiente paso.

N.º	Consulta o prompt	Objetivo	Información obtenida
1	«Cómo se utiliza un array para crear una base de datos de acuerdo a la estructura que ya tienes y me salen estos errores»	Entender el array e identificar errores.	Se revisó bdtemporal.ts, la clase Temporal, los campos id, nombre y edad, y una declaración duplicada de usuarios.
2	«Este es mi usuario service ts»	Revisar el servicio existente.	Se identificaron create, findAll, findOne, update y remove, que inicialmente devolvían mensajes de texto.
3	«Esto es lo que tengo en el DTO: export class CreateUsuarioDto {} y me salen errores en el usuario service porque no están declarados»	Definir los datos que necesita el servicio.	Se propuso declarar nombre y edad en CreateUsuarioDto y hacer opcionales esos campos en UpdateUsuarioDto.
4	«Me salen estos errores» (captura de VS Code)	Resolver los errores que quedaban en el DTO.	Se explicó el uso de nombre!: string y edad!: number para indicar que los valores se asignarán después.
5	«Ya está en cero problemas, ahora dime qué hago para verificar que está correcto»	Comprobar que el proyecto funciona.	Se indicó ejecutar npm run start:dev y probar las rutas de la API.
6	«Me salió esto» (captura de la terminal)	Interpretar el resultado del inicio del servidor.	La terminal mostró cero errores de compilación, inicio correcto de NestJS y registro de rutas GET, POST, PATCH y DELETE.
7	«Estoy probando en Postman»	Continuar las pruebas en Postman.	Se explicaron GET /usuario, GET /usuario/2 y POST /usuario con un cuerpo JSON.
8	«Ahora dime cómo hago el patch y el delete»	Aprender a modificar y eliminar usuarios.	Se explicó PATCH /usuario/4 y DELETE /usuario/4, con una comprobación posterior mediante GET.
9	«¿Se puede hacer con head y options, dime para	Conocer otros métodos HTTP.	Se explicó que HEAD devuelve cabeceras sin el cuerpo y OPTIONS permite consultar

	qué sirven?»		opciones de comunicación admitidas.

3. Cómo avanzó el proceso

Primero: revisar la estructura. Se revisó bdtemporal.ts y cómo guardar usuarios en un array.

Segundo: corregir los DTO. Se declararon nombre y edad para que el servicio pudiera utilizar esos campos.

Tercero: iniciar el servidor. Se ejecutó npm run start:dev y se comprobó que NestJS iniciara sin errores de compilación.

Cuarto: probar el CRUD. Se prepararon solicitudes de Postman para consultar, crear, modificar y eliminar usuarios.

Quinto: aprender otros métodos. Se revisaron HEAD y OPTIONS y sus diferencias con los métodos CRUD.

4. Palabras clave

Palabra clave	Explicación sencilla
Array	Lista que guarda varios usuarios temporalmente.
Temporal	Clase que define los datos de cada usuario.
DTO	Clase que define los datos recibidos al crear o actualizar un usuario.
Service	Archivo donde se realizan las operaciones sobre los usuarios.
Controller	Archivo que recibe solicitudes HTTP y llama al servicio.
CRUD	Crear, leer, actualizar y eliminar datos.
Postman	Herramienta para enviar solicitudes y revisar respuestas de una API.
GET	Consultar información.
POST	Crear un registro.
PATCH	Modificar parte de un registro.
DELETE	Eliminar un registro.
HEAD	Consultar cabeceras sin recibir el cuerpo de la respuesta.
OPTIONS	Consultar las opciones de comunicación permitidas por una ruta o servidor.
CORS	Reglas para solicitudes entre distintos orígenes desde un navegador.

5. Resultados obtenidos

- Se corrigieron los problemas de compilación hasta que VS Code mostró cero problemas relacionados con los errores revisados.
- Se preparó un array temporal con tres usuarios iniciales: Juan, María y Pedro.
- Se adaptó el servicio para trabajar con operaciones CRUD.
- La terminal confirmó que NestJS inició correctamente y registró las rutas de usuario.
- Se recibieron instrucciones para probar GET, POST, PATCH y DELETE en Postman.
- Se aprendió para qué sirven HEAD y OPTIONS.

6. Observación importante

El array guarda datos en la memoria del servidor. Si se detiene o reinicia el servidor, los cambios hechos en Postman se pierden y vuelven a aparecer los datos iniciales definidos en bdtemporal.ts. Aunque la terminal confirmó que el servidor inició y registró las rutas, la conversación no incluyó capturas de las respuestas finales de todas las pruebas de Postman; por eso no se puede afirmar que todas las operaciones se verificaron completamente.

7. Conclusión

El proceso pasó de corregir errores de TypeScript a preparar pruebas de una API REST con NestJS. Se revisó cómo un array puede guardar datos temporalmente y cómo Postman permite probar las operaciones de una API. Si se necesita conservar los datos después de reiniciar el servidor, el siguiente paso sería conectar el proyecto a una base de datos real.